

Meeting 10/6

Student can retry every time they submit an answer, they just see if they're right or wrong

Eventually the student can submit and then it gets graded, plus the student sees the answers

ONLY show last 25 questions in the request help page

Add a limit to the number of questions students can generate (25 new questions per meeting)

Customer feedback:

David Galullo:

Summary is good

Exercises are a struggle

Consistently inconsistent

Format of answers is bad - eg, when you conjugate a verb, it marks you wrong because you don't use the preposition sometimes.

Often times it will ask you a verb to conjugate, other times for fill in the blank you have to guess what to do (can use any word maybe, so which one to use?) - prompt changing

Grading can't handle synonyms

Requesting help doesn't give you the questions you actually just answered

Summary: Grading, and exercises that are returned when asking for help is wrong

Prompts probably need reworking

Katrine Gray:

Top 2-3 problems:

- Sometimes when exercises are requested the software gets confused. Suddenly thinks she's trying to learn english (?). She got exercises in English, as if she is an Italian student learning English.
 - Has something to do with AI prompt when the transcript was generated
- UI/UX sucks for new exercises - students can't understand where to submit answers

What you like:

- Very useful for Italian learners
- Summary molto bene
- Assessing what was done right vs wrong

Olga meeting:

What I did:

- Postman setup was wrong so I spent a while fixing that
- Started using ngrok

- learn about Django orm
- Made a schema to collect student answers
- Migrated it properly using Django to make the table on supabase
- Made the endpoint to ingest and upload it on the backend
- Also checks if answers are correct or wrong
- Tested it
- Made changes on the frontend for this
- Made an endpoint to return the latest answers by the student

Need to figure out/do:

- Migrate old exercises to the new db (done)
- Finish exercise flow entirely (almost done - good enough for now)
 - figure out how to show student feedback to professor (Junge)
 - save this feedback to the db (not needed? DO EVENTUALLY)
 - show how they did on the frontend after clicking submit (done)
 - Handle payload refreshing when submit button is clicked (done)
- Code review
- Merge into dev or main (backend)
- Merge into dev or main (db orm module)

Olga meeting 10/13

I want to get our opinion on Valerio's concerns about the milestones and understand how the work is distributed in the team.

Early on, we were supposed to be learning the codebase. Since I seemed the most familiar with it and felt like we needed to start on features, I just started working on the first set of features related to exercise generation.

It felt like Deniz especially was making very little progress in understanding the code. Junge didn't seem to give that much time to the project and both were sick for a while or otherwise busy, so I've ended up doing most of the work so far.

Eventually since I wanted some help on implementing these features I tasked Junge with some stuff related to the feature. He seems capable but doesn't give that much time to the project and seems to prioritize minimizing work over doing things well.

They both seem kind of demoralized about the project. It's kind of hard to get them motivated to work on stuff.

Deniz started working on the frontend design and is making SQL queries for the tracking portion. I think he'll be sticking to that mostly going forward.

Going forward, Junge will be looking into the AI P2P stuff more because he seems interested in that.

Concerns:

The undergrad team doesn't seem to know what they are doing. (Example about email verification thing). I feel like I have to watch what they do now in case they break something. I don't really want to mentor 4 undergraduates. I think it would be nice if they spent some more time understanding and learning instead of complaining.

To do:

Split up work and assign tasks for the next two weeks ahead of time.

Walk Deniz through the backend

To do for 10/15:

- figure out bug on frontend with correct/incorrect answers (done)
- Remove old code (done, except pipeline.py)
- Add documentation (done)
- Push final backend changes (done)
- Merge code into main branch (done)
 - Make copilot changes (done)
 - Stop using feedback blob (done)
 - Adjust responses to return errors (done)
- Verify deployment worked
- Connect old pages to new ones on FE (done)
- Deploy pages on FE (done)
- Switch to new api on front end (done?)
- Mark as done on notion and GitHub
- Test on main website (done)

Meeting 10/17:

Work done:

Fixed GitHub branch differences between main and dev

Made a few PR's to fix it

Adjusted code review according to copilot suggestions (some were wrong)

Merged my own code into main

Changed frontend API's

Changed old pages to point to new ones and tested that

Website broke from deployment, so I found and fixed a bug

Tested functionality that was deployed

Worked on slides

- made system diagrams
- Made schema diagrams

Did code review for undergrads and looked at subsequent changes too

To do:

Work on frontend for infinite exercise generation

Start on backend for it
Work on midterm presentation

Olga midterm meeting:
Don't go super low level, but go low level

To do 10/22:

- Fix endpoint so it doesn't point to my backend and deploy
- Continue work on backend and frontend for infinite exercise generation
 - MAKE ISSUE
 - Finish main frontend page
 - History frontend page
 - Endpoint to generate and store exercises under the "infinite" category for student responses and for exercises
- Fix exercise flow:
 - Issue?
 - Design a page to show existing exercises WITHOUT new generation (basically a history page) - could be same as for infinite
 - Fix bug with frontend where correct/incorrect does not update
 - Figure out how to categorize it (change prompt). If student wants n questions, what type should it be?
 - UI fixes?
- Work on tracking system on the backend
 - Make issue
- Work on issues related to summary generation and pipeline failures (this is unrelated to our code; Valerio asked us to look at it)
 - Look at logs
 - Check prompt generation
-

To delegate:

- Who finishes frontend for teachers?
- Tracking completion for any student
 - Fix how this is tracked in db?
- Deniz - make an issue please

Sort historic exercises by date
Show 10-20 per page and paginate (?)

10/27:

I need to do:

2. Make an issue for infinite exercises

3. Db changes:
 1. Adjust exercises table so foreign keys can be null
 2. Same for student responses
4. Backend:
 1. Adjust backend to use user_id from the frontend to store and retrieve exercises with inf-ex and user id's
 1. Generation
 2. Storage of responses
 3. Retrieval of responses
 4. Retrieval of exercises
 2. Test it
5. Frontend:
 1. Adjust payloads to send back the user_id and verify from frontend
 1. Display_sli going into generated exercises
 2. Dashboard going to exercise history
 3. Meeting going to exercise history
 4. Dashboard going to re-answer old questions
 5. Meeting going to re-answer old questions
 6. Dashboard access point to -> generating infinite
 7. When retrieving student responses (curr method)
 8. When updating student responses (inf and sli pages)
 9. When generating more questions from the infEx page
 2. Design the exercise history page - just show 100 exercises for now
 3. Add connections to exercise history and answer page from:
 1. Dashboard for infinite exercises
 2. Meetings for meeting-specific exercises
 4. Make it possible to generate even more exercises from the same page
 5. Figure out how to paginate the exercise history page
 6. Confirm the infinite exercise page stores exercises properly in the db
 7. Figure out how to get categories associated with generated exercises

Issues:

- How to get user ID?
 - It's stored in the frontend after the user logs in
- Students don't need a page to just re-answer old questions
 - Maybe allow them to

Exercise flow:

Tell students on the page that questions they don't answer don't count against them

Allow students to re-answer

Allow students to see response history

Allow students to

Olga meeting. What I did:

- Confirmed the flow and everything for infinite exercises to Valerio
- Made issues for infinite exercises in both db's
- reworked backend to handle infinite exercises
 - Reworked 4 endpoints for it
 - Tested it out a bunch
 - Tested frontend api calls to it
- Reworked db for the same
- Learned new stuff about constraints
- Made the frontend page changes required

To do:

- Connect frontend changes to represent flow
- Test it out once more
- Make PR's and deploy
- Review Deniz's code

To do 11/3:

- find out where bug occurs in code related to student responses
- Rework frontend after that
- Make additional frontend fixes

Fixing response bug. Current issue:

1. Time is sent back incorrectly
2. Frontend payload sends back student responses within the "responses" key. The value of this key is a dict with the following format:

```
"responses": {  
    "question_uuid_1": {  
        "student_response": "student's answer to question 1",  
        "time": "time taken to answer question 1"  
    },  
    ...,  
    "dummy_value": "" # This is a dummy value to handle the trailing  
comma created in the response from the frontend when the user submits their  
answers  
}
```

This is a problem because students can answer the same question multiple times before submitting. So if they answer the same question, only one (the first key) gets processed.

Solution:

Convert it to a list of responses.

New Format (Preserves All Responses)

```
{
  "meeting_uuid": "",
  "infinite_exercises": "True",
  "user_id": "LH03UIFUU5",
  "responses": [
    {
      "question_uuid": "a8e233e0-47cc-4a8b-aa5f-4d68c29d648f",
      "student_response": "test again",
      "time": "2025-11-03 18:11:08.03+00:00"
    },
    {
      "question_uuid": "a8e233e0-47cc-4a8b-aa5f-4d68c29d648f",
      "student_response": "scrive",
      "time": "2025-11-03 18:11:13.66+00:00"
    },
    "dummy_value": ""
  ]
}
```

Workflows changed:

1. Page load payload construction - change `"responses": {}` to `"responses": []`
2. Change workflow for when user answers something:
 1. Instead of appending

```
"question_uuid_1": {
  "student_response": <student's answer to question 1>,
  "time": <time when answer 1 was entered>
},
append
{
  "question_uuid":<question_uuid>,
  "student_response": <student's answer to question 1>,
  "time": <time when answer 1 was entered>
},
```

2. Adjust datetime - change it to `yyyy-mm-dd'T'HH:MM:ss.'Z'` as the custom format. This incorporates the time correctly.

1. Change the submit button workflow:
 1. Append `{"dummy_value":""}]}` instead of what exists

To do:

- ☐ Watch that meeting recording
- ☒ Push backend changes
- ☒ Make backend issue
- ☒ Make new frontend for sli ex
- ☒ fix frontend for inf ex

Meeting 11/3 notes:

Look into why exercises don't get generated

Show the previous answer above the answer - make it possible for students to toggle it

Change page title for exercise history and other pages, it just shows "Infinite Exercises" now

When viewing exercise history - make sure it is filtered correctly and shows the correct exercises for each page. Right now, it doesn't filter correctly. It needs to be confirmed for:

Infex:

- Check old ones
- Create new ones
- View old responses

SLI:

- Check old ones
- Create new ones
- View old responses

How do you limit it to 25 per SLI?

- Change the exercises table to accept a flag that says whether it was generated when the transcript arrived
- Db:
 - Add field to exercise table in schema called "generated_by_student"
 - Flag everything as false when migrating
 - Do not migrate until backend is done
- Backend:
 - Set generated_by_student to True when student generates an exercise from the backend
 - If the count of exercises associated w a meeting that is generated_by_student is > 25, return a response saying more aren't allowed
 - Otherwise, generate min(25-num(generated_by_student), num_questions)
 - Set generated_by_student to False when exercises generated upon

transcript retrieval

- This can be used to figure out how many questions have been generated for meeting
- Then, the response can be adjusted to figure this out
- The limit is only for per meeting
- **Show some popup when 25 exercises have been generated**

Look into prompts for generating questions

- Make sure it's generating the correct number of questions
- Figure out what context it includes
- Include the transcript AND exercises, or only the transcript, for query context
- Categorize it somehow?
 - How to split it up between categories?

AI p2p:

- A student sees all other tier 2 users
- They mutually agree?

Students whose exercises are weird:

Kris

Rita

Sophia

Figure out why their exercises are in a different language

- This is a problem only for *new* students
- It's because there was a failure in their initial transcript-arrival exercise generation
- When including previous exercises, there were none to show
- For other students, they had some italian ones
- So the default language of 'en' is used, as the target language isn't even passed
- When testing, test with THOSE students

Change the "Practice Exercises" to "Infinite Exercises"

One issue for limiting to 25

- do the filtering correctly part here (this is already correct)

Issue: limit exercises generated by students to 25 per session

Right now, students can generate as many exercises as they want for a particular session. solving this issue will restrict this to 25 allowed questions per session per student.

Proposed changes:

- Add a flag to the exercises table to see if a question was generated by a student for a session

- Set this to false as default for now, so going forward, students can only generate 25 new questions for each session
- When students try to generate questions, the backend will check if they have 25 generated questions for this. If so, it will not generate more
- If a student has n questions left, and they want to generate x questions, the number of questions generated will be $\min(n, x)$
- **IMPORTANT:** Make sure to filter by user id AND session, not just session

One issue for fixing badly generated exercises

- also correctly filter exercises
- Check the frontend to ensure the correct params are being passed to the back

Title: fix generated questions not being in the correct language

Currently, students sometimes get questions generated in English instead of their target language.

To do 11/10:

- **For a meeting with multiple students, when one student generates exercises, other students can see it too (eg: Kris & Isabel)**
 - Maybe bc it's not being filtered by meeting and user id?
 - Yeah that's it
 - Check frontend payloads to backend for this
- ~~Make the pop-up for the frontend when a student reaches 25 question limit~~
- ~~Display the student's previous answer along with whether it's correct or not on bubble~~
- Get meeting dates and user names to re-process transcripts for erroneous ones

Issue for exercise generation prompt:

To do:

1. Fix Incorrect parser
 2. Include transcript in prompt OR parts of the SLI
 3. Associate a category with each question - it has to be parseable
 4. What should be done with the existing categorized questions if the new ones are different?
- Question: If the LLM generates extra questions, should I ignore the extras or keep them?

- Throw away the extras
- Question: There are many types of exercises the llm can generate. Which ones should I focus on? T/F? Fill in the blank? Translation?
 - Categories:
 - True/False
 - Fill in the blank
 - Translation

Big error for meetings with multiple participants - generated and uploaded exercises are all tied to the first non-teacher participant in a meeting. How to do it for each student?

- Maybe repeat it for each student

Checklist:

categorization:

- Include transcript
- Confirm transcript is included
- Confirm logic for when transcript is used
- Use Pydantic objects to parse a response
-

multiple participants:

- Go over code and make sure it's ok

To fix:

- Remove the exercise generation process when the transcript arrives
- Fix the 'exercises not generated' placeholder coming through
 - Committed
 -
- Fix popup not showing up
- Change dropdown options if student has already generated questions
 - Matt said something like just give the number of questions left?
 - Or just generate all 25 questions at once all the time
 - Or grey it out when it's ≥ 25 exercises
 - Best idea: Add a disclaimer that the llm may not generate exactly the number of exercises
 - Allow them to click the generate button until they reach or cross 25
 - Once it's ≥ 25 , just remove those buttons and add a message that they've reached their limit
 - Send 25 back from the frontend when generating questions
- Done:
 - Added an endpoint that returns num of generated questions for a

student

- If this is ≥ 25 , the buttons do not show
- Todo: Deploy frontend
- Todo: Deploy backend
- Todo: Change url of frontend api connector to point to hosted endpoint

-

12/8/25 notes:

Test gpt-4.1 once more tomorrow and then push

Prepending instructions may no longer be needed

Push changes to fix student pipeline

Pitch:

Imagine a typical online language lesson: students meet for an hour, practice, get corrections, and make progress, but as soon as the call ends, most of that value disappears. They can't easily revisit what they learned, as it is difficult to both take comprehensive notes while learning and speaking the language, they don't always get exercises tied to their lesson, and they rarely have a structured way to practice with others at their level. Teachers, on the other hand, may have dozens of students. They often lack a simple, clear way to see how their class is doing beyond gut feeling, and managing several students means it is harder to remember what occurred in prior lessons, as each student is on their own path. Duets.AI was built to fix these gaps. It's a companion platform that strengthens learning between lessons by using AI, not by replacing teachers, but by extending their reach and giving students more meaningful ways to practice.

Here's how it works. As you can see in the diagram on the bottom left, after a lesson is started on Zoom, webhooks trigger our backend to record the meeting's beginning. Once it ends and the recording is available, webhooks trigger the transcription process in our backend to convert the meeting audio into text. From there, we have specific prompts used to study the conversation - figuring out where students stumbled, where they did well, what themes came up, and what skills were actually used. Using this, Duets.AI produces four key features to support the learning and teaching experience

1. **Learning insights for each student and teacher** - a comprehensive summary of what the lesson covered, tailored differently for each student and teacher.
2. **Targeted exercises** - students can generate exercises which are tailored to their own mistakes and successes, and pulls directly from the transcript of the meeting.
3. **Dashboards** - crisp, high-level overviews that show teachers how their class is doing, and students how they are doing.

4. AI-assisted Peer-to-Peer chat - one of the best ways to learn a language is to talk with your peers in it. We built a feature that uses an LLM as a moderator as students chat with each other in the language they are learning together.

The exercises, dashboards, peer-to-peer chat, and redesign of the frontend comprise the majority of features we developed over this semester.

In the middle, you can see high level flow diagrams of our system and data. We developed our backend in Python and use Django to orchestrate both the backend and interaction with the database on Supabase. When we push new features, github actions automatically triggers a workflow that containerizes our code and deploys it to Azure. Azure also hosts our nginx reverse-proxy, allowing our backend to communicate with the frontend and the internet. Our frontend is built using bubble.io, a no-code frontend builder. Apart from our transcription service, which is hosted in its own container, all other backend and AI functionality takes place in our analysis service. We use LangChain to talk to OpenAI's ChatGPT to make summaries and exercises. We also use Stripe for payment processing. Behind the scenes, the team faced real engineering challenges: connecting the no-code frontend to the backend services, designing a scoring system that feels fair and informative, reshaping parts of the interface in Bubble, and updating system design to better suit scalability and feature development. Through close collaboration among developers, sponsors, and the undergraduate contributors, we ultimately delivered a product we can be proud of.

